

Rajalakshmi Engineering College

Name: M.S Nanthana shree
Email: 241901064@rajalakshmi.edu.in
Roll no: 241901064
Phone: 9360753427
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units 4 per unit For the next 100 units (51–150) 6 per unit
For units above 150 8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

Answer

```
import java.util.Scanner;
```

```
class GasCustomer {
    private int customerID;
    private String customerName;
    private double unitsConsumed;

    public GasCustomer(int customerID, String customerName, double
unitsConsumed) {
        this.customerID = customerID;
        this.customerName = customerName;
        this.unitsConsumed = unitsConsumed;
    }

    public int getCustomerID() {
        return customerID;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double calculateBill() {
        double units = unitsConsumed;
        double bill = 0;

        if (units <= 50) {
            bill = units * 4;
        } else if (units <= 150) {
            bill = 50 * 4 + (units - 50) * 6;
        } else {
            bill = 50 * 4 + 100 * 6 + (units - 150) * 8;
        }

        if (bill > 2000) {
            bill *= 0.85; // 15% discount
        }

        return bill;
    }
}

public class Main {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();

    for (int i = 0; i < N; i++) {
        int customerID = sc.nextInt();
        String firstName = sc.next();
        String lastName = sc.next();
        String customerName = firstName + " " + lastName;
        double units = sc.nextDouble();

        GasCustomer customer = new GasCustomer(customerID, customerName,
units);
        double finalBill = customer.calculateBill();

        System.out.printf("Customer ID: %d Customer Name: %s Final Bill: %.1f\n",
            customer.getCustomerID(), customer.getCustomerName(), finalBill);
    }

    sc.close();
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are working as a developer for CityMobile, which wants to build a basic mobile data usage management system.

Each customer has:

A Customer ID (integer) A Customer Name (string) An Initial Data Balance (in GB, double)

The company allows two types of operations:

Recharge – increases the data balance. Usage – decreases the data balance only if enough data is available.

If the usage amount is greater than the available data balance, the usage

should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details after all operations.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Initial Data Balance (double).
- The next line contains the Recharge Amount in GB (double).
- The next line contains the Usage Amount in GB (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Data Balance: <final_data_balance> GB (The final balance must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Ravi Kumar

5.0
2.0
3.0

Output: Customer ID: 1234
Customer Name: Ravi Kumar
Final Data Balance: 4.0 GB

Answer

```
import java.util.Scanner;
```

```
class MobileCustomer {  
    private int customerID;  
    private String customerName;  
    private double dataBalance;  
  
    public MobileCustomer(int customerID, String customerName, double  
dataBalance) {  
        this.customerID = customerID;  
        this.customerName = customerName;  
        this.dataBalance = dataBalance;  
    }  
  
    public int getCustomerID() {  
        return customerID;  
    }  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public double getDataBalance() {  
        return dataBalance;  
    }  
  
    public void recharge(double amount) {  
        if (amount > 0) dataBalance += amount;  
    }  
  
    public void useData(double amount) {  
        if (amount > 0 && amount <= dataBalance) dataBalance -= amount;  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        for (int i = 0; i < N; i++) {
            int customerID = sc.nextInt();
            String firstName = sc.next();
            String lastName = sc.next();
            String customerName = firstName + " " + lastName;
            double initialBalance = sc.nextDouble();
            double rechargeAmount = sc.nextDouble();
            double usageAmount = sc.nextDouble();

            MobileCustomer customer = new MobileCustomer(customerID,
customerName, initialBalance);
            customer.recharge(rechargeAmount);
            customer.useData(usageAmount);

            System.out.printf("Customer ID: %d Customer Name: %s Final Data
Balance: %.1f GB\n",
                customer.getCustomerID(), customer.getCustomerName(),
customer.getDataBalance());
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed

(double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per liter
For the next 500 liters (501–1000) 3 per liter
For liters above 1000 5 per liter
If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

Answer

```
import java.util.Scanner;
```

```
class WaterCustomer {  
    private int customerID;  
    private String customerName;  
    private double litersConsumed;  
  
    public WaterCustomer(int customerID, String customerName, double  
litersConsumed) {  
        this.customerID = customerID;  
        this.customerName = customerName;  
        this.litersConsumed = litersConsumed;  
    }  
  
    public int getCustomerID() {  
        return customerID;  
    }  
  
    public String getCustomerName() {  
        return customerName;  
    }  
  
    public double calculateBill() {  
        double liters = litersConsumed;  
        double bill = 0;  
  
        if (liters <= 500) {  
            bill = liters * 2;  
        } else if (liters <= 1000) {  
            bill = 500 * 2 + (liters - 500) * 3;  
        } else {  
            bill = 500 * 2 + 500 * 3 + (liters - 1000) * 5;  
        }  
    }  
}
```

```

        if (bill > 3000) {
            bill *= 0.9; // 10% discount
        }

        return bill;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        for (int i = 0; i < N; i++) {
            int customerID = sc.nextInt();
            String firstName = sc.next();
            String lastName = sc.next();
            String customerName = firstName + " " + lastName;
            double liters = sc.nextDouble();

            WaterCustomer customer = new WaterCustomer(customerID,
customerName, liters);
            double finalBill = customer.calculateBill();

            System.out.printf("Customer ID: %d Customer Name: %s Final Bill: %.1f\n",
customer.getCustomerID(), customer.getCustomerName(), finalBill);
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:

Runner ID (integer) Runner Name (string) An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5). Identify the fastest runner (the one with the lowest average time). If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details. A constructor to initialize runner details. Getter and Setter methods to retrieve and update runner details if required. A method to calculate the average time. Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

Input Format

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

Output Format

For each runner the output prints the following details:

- Runner ID: <runner_id>
- Runner Name: <runner_name>
- Average Time: <average_time>

Finally, print "Fastest Runner: <runner_name> with <average_time> minutes"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

240 250 245 255 260

Output: Runner ID: 1001

Runner Name: Ravi Kumar

Average Time: 250

Fastest Runner: Ravi Kumar with 250 minutes

Answer

```
import java.util.Scanner;
```

```
class Runner {
```

```
    private int runnerID;
```

```
    private String runnerName;
```

```
    private int[] times;
```

```
    private int averageTime;
```

```
    public Runner(int runnerID, String runnerName, int[] times) {
```

```
        this.runnerID = runnerID;
```

```
        this.runnerName = runnerName;
```

```
        this.times = times;
```

```
        calculateAverageTime();
```

```
    }
```

```
    public int getRunnerID() {
```

```
        return runnerID;
```

```
    }
```

```
    public String getRunnerName() {
```

```
        return runnerName;
```

```
    }
```

```
    public int getAverageTime() {
```

```

        return averageTime;
    }

    private void calculateAverageTime() {
        int sum = 0;
        for (int time : times) sum += time;
        this.averageTime = sum / times.length;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();

        Runner[] runners = new Runner[N];

        for (int i = 0; i < N; i++) {
            int runnerID = sc.nextInt();
            String firstName = sc.next();
            String lastName = sc.next();
            String runnerName = firstName + " " + lastName;
            int[] times = new int[5];
            for (int j = 0; j < 5; j++) times[j] = sc.nextInt();

            runners[i] = new Runner(runnerID, runnerName, times);
        }

        Runner fastestRunner = runners[0];

        for (Runner r : runners) {
            System.out.printf("Runner ID: %d Runner Name: %s Average Time: %d\n",
                r.getRunnerID(), r.getRunnerName(), r.getAverageTime());

            if (r.getAverageTime() < fastestRunner.getAverageTime() ||
                (r.getAverageTime() == fastestRunner.getAverageTime() &&
                r.getRunnerID() < fastestRunner.getRunnerID())) {
                fastestRunner = r;
            }
        }

        System.out.printf("Fastest Runner: %s with %d minutes\n",

```

```
        fastestRunner.getRunnerName(), fastestRunner.getAverageTime());  
    }  
    sc.close();  
}
```

Status : Correct

Marks : 10/10